

I want to formalize a log t temporal consolidation variant of VAMP, based roughly on the following notes:

4. A logarithmic stack of temporal LoRAs

Your logt idea fits naturally as an LSM-tree-like hierarchy.

For each batch of m examples:

Show more ▾

Worked for 1m 43s >

Yes. I think there is a clean first formulation here, and it is simpler than the existing VAMP addressing machinery.

The core trade is:

Instead of retaining many fine-grained adapters and solving a hard retrieval problem, continually compress history into $O(\log T)$ temporal adapters and simply evaluate all of them.

This fits the addressed-parameter-memory view: the frozen base remains the root and the temporal LoRAs are the currently reachable parameter deltas; what changes is that the memory structure itself guarantees a tiny address set.  Addressed_Parameter_Memories

1. Basic setup

Let the frozen base parameters be

$$\theta_0.$$

Training data arrive in batches

$$B_1, B_2, \dots, B_T, \quad |B_t| = m.$$

A level- j temporal chunk

$$C = (j, I, \phi)$$

contains:

- a contiguous interval I of 2^j input batches,
- hence a dataset D_I containing $2^j m$ examples,
- a standalone LoRA adapter Δ_ϕ trained relative to the same frozen base θ_0 .

Its model is therefore

$$\theta_C = \theta_0 + \Delta_\phi.$$

That last point is important: **chunks should be base-relative, not chained residuals** in this first version. Then every temporal chunk can be evaluated independently against a query.

2. Use two chunks per level

I would slightly modify your binary-counter formulation.

Set a small fixed level capacity

$$b = 2.$$

Each level may contain at most two chunks. When inserting a third chunk at level j , merge the **two oldest** into a level- $j + 1$ chunk.

So:

1. Train a level-0 adapter on B_t .
2. Insert it into level 0.
3. If level 0 now contains 3 chunks, merge its two oldest chunks.
4. Insert the result into level 1.
5. Recursively repeat if that level overflows.

The merge operation is

$$\text{Merge}(C_a, C_b) = \text{TrainLoRA}(\theta_0, D_a \cup D_b),$$

where both children are level j , so the result covers

$$|D_a \cup D_b| = 2^{j+1}m$$

examples.

The children then disappear from the **active adapter set**.

Why two rather than one?

With one chunk per level, at $T = 16$ the entire history becomes one level-4 adapter. You technically have $O(\log T)$ storage, but have lost essentially all temporal addressing resolution.

With two chunks per level, at $T = 16$ the active intervals look roughly like

$$[1, 8], [9, 12], [13, 14], [15], [16].$$

That is exactly the shape we want:

fine resolution for recent experience; progressively coarser resolution into the past.

It is still only $O(\log T)$ adapters.

For $T = 13$, for example:

$$[1, 4], [5, 8], [9, 10], [11, 12], [13].$$

No semantic indexing is required to decide which of these matters.

3. The important invariant

Let $\mathcal{H}_T$ be the set of active chunks after T input batches.

Every chunk at level j represents exactly

$$2^j$$

original batches, and there are at most b chunks at each level.

Since there cannot be a chunk above level

$$J = \lfloor \log_2 T \rfloor,$$

we immediately have

$$|\mathcal{H}_T| \leq b(1 + \lfloor \log_2 T \rfloor).$$

For $b = 2$,

$$|\mathcal{H}_T| = O(\log T).$$

This is the critical property. It makes the addressing problem qualitatively different from ordinary VAMP.

4. Exhaustive addressing

For query x , define the candidate parameterizations

$$\Theta_T = \{\theta_0\} \cup \{\theta_0 + \Delta_C : C \in \mathcal{H}_T\}.$$

Let $A(x, C)$ be whatever VAMP uses to score how well adapter C explains or predicts x . For an LM this could initially just be prompt NLL or a related held-out predictive score; for an energy-based implementation it can be the settled energy.

Then simply compute

$$C^*(x) = \arg \min_{C \in \mathcal{H}_T \cup \{0\}} A(x, C).$$

There is no ANN lookup, Hopfield retrieval, learned router, gradient index, or hierarchical search.

You just try them all.

Since

$$|\mathcal{H}_T| = O(\log T),$$

addressing requires

$$O(\log T)$$

adapter evaluations.

More precisely, if one model evaluation costs C_{eval} ,

$$C_{\text{address}} = O(C_{\text{eval}} \log T).$$

Those evaluations can also be batched across adapters, although that changes wall-clock cost rather than the asymptotic work.

This is the main conceptual win

The original VAMP formulation says roughly:

large parameter memory + efficient address search.

This version says:

aggressively compressed parameter memory + exhaustive address search.

So we have traded the addressing problem for a consolidation problem.

That seems like a very worthwhile trade to investigate.

5. Consolidation complexity

Suppose processing one training example for a fixed number of epochs costs $O(1)$ in units of model-training work.

A level- j merge processes

$$2^{j+1}m$$

examples, so its cost is

$$O(2^j m)$$

up to a constant factor.

But such a merge can happen only after roughly 2^j new batches have accumulated.

Therefore the amortized contribution of level j per incoming batch is

$$\frac{O(2^j m)}{2^j} = O(m).$$

There are only

$$O(\log T)$$

occupied levels. Hence

$$C_{\text{update}}^{\text{amortized}} = O(m \log T).$$

Equivalently, after T batches,

$$C_{\text{total}} = O(mT \log T).$$

There is an even simpler way to see the same result:

Each example can participate in at most one consolidation at each level, and there are only $O(\log T)$ levels.

So every example is trained on at most $O(\log T)$ times.

6. Worst-case latency is different

The amortized result should not be confused with worst-case insertion latency.

If an insertion triggers a long chain of carries, a single update can require

$$O(m) + O(2m) + O(4m) + \dots + O(Tm) = O(Tm)$$

work.

So the synchronous algorithm has

$$C_{\text{update}}^{\text{worst}} = O(Tm).$$

This doesn't break the amortized result.

And there is a straightforward deamortized version: treat each level merge as an incremental job and give each level $O(m)$ training work per incoming batch. A level- j job has $O(2^j m)$ work and receives approximately 2^j arrivals before another same-scale job is required, so it can in principle be completed in time.

That gives a route toward

$$O(m \log T)$$

scheduled consolidation work per arrival without latency spikes, at the cost of temporarily having old and partially consolidated adapters simultaneously.

I would **not make this machinery part of v1**, though. Prove the amortized version first.

7. Adapter-memory bound

Let a LoRA chunk contain p_A stored parameters.

Because there are $O(\log T)$ active chunks,

$$M_{\text{adapters}} = O(p_A \log T).$$

This is unusually attractive for VAMP because both:

- search complexity, and
- live parameter-memory complexity

grow logarithmically with lifetime.

There is an important caveat:

Training-data storage is separate

If merging two chunks means retraining on **all examples represented by those chunks**, then we need to retain those examples—or something equivalent.

Exact source-data retention costs

$$O(mT)$$

data memory.

So the initial theorem should explicitly say:

$$\text{active adapter storage} = O(p_A \log T),$$

while

$$\text{archival training-data storage} = O(mT).$$

Getting the latter below linear requires coresets, generative replay, sufficient statistics, etc. That is a separate problem and should not be smuggled into this first formulation.

8. Don't solve compressibility yet

I agree with your instinct here.

In fact, for the first version I would remove

“attempt to consolidate”

from the algorithm entirely.

Instead:

Always perform the scheduled consolidation, measure its distortion, and record the result.

That lets the experiment tell us when temporal LoRA compression breaks down.

Later, a merge acceptance policy can use that measurement.

We only need one definition now so we know what we're measuring.

Suppose children C_a, C_b have datasets D_a, D_b , and their specialized losses are

$$L_{D_a}(C_a), \quad L_{D_b}(C_b).$$

For proposed merged chunk C' , define its merge distortion as

$$\delta(C') = \max [L_{D_a}(C') - L_{D_a}(C_a), L_{D_b}(C') - L_{D_b}(C_b)] .$$

This is deliberately boring.

It simply asks:

How much worse did replacing these two adapters by one adapter make each region of data?

We don't yet say that $\delta < \epsilon$ means “compressible,” or what to do if it isn't.

Just measure δ .

9. This immediately gives us an eventual error theorem

The definition above also sets up a useful result without requiring us to solve the merge-policy question.

Suppose a batch B_s starts with level-0 adapter C_s^0 . Over time it participates in merges at levels

$$0, 1, \dots, J - 1.$$

If the loss increase attributable to the merge at level j is bounded by

$$\epsilon_j,$$

then the final temporal chunk C_s^J containing B_s satisfies

$$L_{B_s}(C_s^J) \leq L_{B_s}(C_s^0) + \sum_{j=0}^{J-1} \epsilon_j.$$

Since

$$J \leq \log_2 T,$$

a uniform merge bound $\epsilon_j \leq \epsilon$ yields

$$L_{B_s}(C_s^J) \leq L_{B_s}(C_s^0) + O(\epsilon \log T).$$

That's not yet a particularly strong CL theorem, but it tells us exactly what we eventually need from consolidation.

For example, if the allowed distortion decreases with scale such that

$$\sum_{j=0}^{\infty} \epsilon_j < \infty,$$

then historical loss degradation stays bounded even as $T \rightarrow \infty$.

That is probably the direction in which the theory eventually wants to go.

But we don't need to choose the ϵ_j schedule yet.

10. Minimal algorithm

Conceptually:

```
state:
    frozen base model  $\theta_0$ 
    levels  $H[0 \dots]$ 
    capacity  $b = 2$ 

on batch  $B_t$ :

     $C \leftarrow \text{TrainLoRA}(\theta_0, B_t)$ 
     $C.\text{level} \leftarrow 0$ 
     $C.\text{data} \leftarrow B_t$ 
    append  $C$  to  $H[0]$ 

     $j \leftarrow 0$ 

    while  $|H[j]| > b$ :

         $A, B \leftarrow$  two oldest chunks in  $H[j]$ 

         $D \leftarrow A.\text{data} \cup B.\text{data}$ 

         $C \leftarrow \text{TrainLoRA}(\theta_0, D)$ 
         $C.\text{level} \leftarrow j + 1$ 
         $C.\text{data} \leftarrow D$ 

        record merge distortion  $\delta(A, B \rightarrow C)$ 

        remove  $A, B$  from  $H[j]$ 
        append  $C$  to  $H[j+1]$ 

     $j \leftarrow j + 1$ 
```

Addressing:

```
ADDRESS(x):

    candidates  $\leftarrow \{\text{base } \theta_0\} \cup$  all active temporal chunks

    for  $C$  in candidates:
         $\text{score}[C] \leftarrow \text{AddressScore}(x, \theta_0 + \Delta_C)$ 

    return  $\text{argmin}_C \text{score}[C]$ 
```

That's almost the entire algorithm.

11. What I would explicitly *not* put into version 1

I would keep the initial scheme extremely constrained:

- no semantic indexing;
- no gradient-index addressing;
- no Hopfield lookup;
- no learned router;
- no adaptive choice of which chunks to merge;
- no merge rejection;
- no clever coresets;
- no adapter composition;
- no variable LoRA rank;
- no dynamically growing adapter capacity;
- no complicated merge initialization.

Use fixed-rank LoRAs and oldest-first temporal compaction.

This gives us a system whose behavior is actually interpretable.

12. One particularly important restriction: single-adapter addressing

Initially I would have the address choose **exactly one** temporal adapter:

$$\theta_{\text{eff}}(x) = \theta_0 + \Delta_{C^*(x)}.$$

Don't initially allow arbitrary combinations like

$$\theta_0 + \sum_C a_C(x) \Delta_C.$$

There are $O(\log T)$ chunks, but arbitrary subset addressing creates

$$2^{O(\log T)} = O(T)$$

possible combinations, immediately throwing away the central advantage.

If composition later proves important, a fixed K -step greedy search would retain

$$O(K \log T)$$

addressing cost.

But single-chunk selection gives us the cleanest experiment.

13. The formal story in four claims

I think the initial proposal can essentially be built around four propositions.

Proposition 1 — Bounded active memory.

For constant per-level capacity b ,

$$|\mathcal{H}_T| \leq b(1 + \lfloor \log_2 T \rfloor) = O(\log T).$$

Proposition 2 — Exhaustive addressing.

Selecting the best active temporal adapter requires

$$O(\log T)$$

candidate evaluations.

Proposition 3 — Amortized consolidation.

If training a chunk containing n examples requires $O(n)$ example-processing work, then inserting a batch of m examples requires

$$O(m \log T)$$

amortized consolidation work.

Proposition 4 — Accumulated consolidation error.

If the loss degradation suffered by a historical example or batch at its level- j consolidation is at most ϵ_j , then after T batches its excess loss due to temporal consolidation is bounded by

$$\sum_{j=0}^{\lfloor \log_2 T \rfloor} \epsilon_j.$$

Those four statements are already enough to make this a distinct and testable VAMP variant.

The deeper interpretation

This version is interesting because it attacks VAMP's addressing difficulty in a fundamentally different place.

Existing VAMP thinking has largely been:

preserve lots of specialized memory → invent a scalable way to find it

whereas this becomes:

continually compress specialized memory → make the remaining address space

The obvious failure mode is equally clear: **temporal consolidation may destroy distinctions that are useful for future queries.**

But that is now essentially the *only* hard empirical question. We no longer simultaneously need a good consolidation method *and* a sublinear approximate addressing system.

I think that's a much better starting point for the next VAMP branch.

Sources



